# Simple Local Search Sets 33 New Records for Balanced Latin Rectangles

Zhengyu Li
Georgia Institute of Technology
brian.li@gatech.edu

Vijay Ganesh
Georgia Institute of Technology
vganesh45@gatech.edu

**Abstract**

A $k \times n$ Latin rectangle is *balanced* when every pair of symbols has the same total distance, summed over the rows. The *imbalance* measures how far a rectangle is from this ideal. Field experiments in agronomy use minimum-imbalance Latin rectangles, and finding them is a benchmark for constraint programming, SAT, and LLM-generated streamliners combined with mixed-integer programming. We show that plain simulated annealing finds rectangles at least as good as all published ones, and better ones on most instances. The search uses an $O(n)$ swap move and an adaptive penalty for column conflicts. It improves the best known imbalance on 33 of the 51 non-square instances with $8 \le n \le 13$ from the literature, and it matches the other 18. The improvements average 18.5% and reach 39.4% ($12 \times 13$: $274.0 \to 166.0$). The search is also faster by orders of magnitude. On one laptop core, it beats each previous record after a median of 0.12 CPU seconds over 5 seeds, and every run beats it within 143 seconds. The previous records came from pipelines that run for at least four hours per instance. For four rectangles, a SAT approach reports solving times from 32 minutes to 5.5 hours. Our search matches each of them in at most 0.07 seconds. We release all rectangles, an independent checker, and the code. The new values are upper bounds, and optimality remains open for every improved instance.

## 1 Introduction

Latin squares and rectangles are classical tools of experimental design. In a field trial, spatial autocorrelation can bias a Latin layout when some pairs of treatments are always planted closer together than others [van Es and van Es, 1993]. *Spatially balanced* Latin squares remove this bias, because every pair of symbols has the same total distance [Gomes and Sellmann, 2004, Le Bras et al., 2012]. Díaz et al. [2017] extended the idea to $k \times n$ rectangles. Balanced rectangles exist only in restricted cases. Díaz et al. therefore defined the *imbalance* of a rectangle and asked for rectangles of minimum imbalance. They found this optimization problem hard for CSP, local-search and MIP methods, and they proposed it as a benchmark.

Two recent studies attacked this benchmark. Peruvemba Ramaswamy and Szeider [2023] used SAT encodings, which also certify optimality with DRAT proofs. Voboril et al. [2025] used streamliners generated by large language models (LLMs) and evaluated them with Gurobi. This approach improved the best known values on most open instances. Balanced Latin rectangles also serve as a test problem for LLM agents that reformulate constraint models [Voboril and Szeider, 2026].

**Contribution.** We revisit the problem with a deliberately simple method. It is simulated annealing [Kirkpatrick et al., 1983] over rows that are permutations, with a swap move that costs $O(n)$ time to evaluate. A penalty for repeated symbols in a column adapts its weight during the search, in the spirit of strategic oscillation [Glover and Hao, 2011]. None of these ingredients is new.

Our contribution is the outcome. We call the best known value of an instance before this work its *record*. On the instances of Voboril et al. [2025], our search improves 33 of the 51 records and is never worse. It is also fast. On a single laptop core, it beats each record after a median of 0.12 seconds, whereas the previous pipelines run for hours per instance. We release every rectangle with an independent checker, so anyone can check each record in seconds without trusting our solver.

We see two consequences. First, the records on this benchmark were much further from optimal than they appeared. This matters to anyone who uses them to judge new methods. Second, new methods for this problem, including LLM-based ones, need a well-engineered local search as a baseline, and not only an exact solver with a time limit.

## 2 Problem

A $k \times n$ Latin rectangle ($k \le n$) is a $k \times n$ array over the symbols $\{1, \dots, n\}$ in which no symbol repeats in a row or in a column. Write $c_i(u)$ for the column of symbol $u$ in row $i$. Following Peruvemba Ramaswamy and Szeider [2023], the distance of two symbols and the imbalance of $L$ are

$$d(u,v) = \sum_{i=1}^{k} |c_i(u) - c_i(v)|, \qquad I(L) = \sum_{1 \le u < v \le n} \left| d(u,v) - \tfrac{k(n+1)}{3} \right|.$$

The target $k(n+1)/3$ is the average of $d(u,v)$ over all pairs, because every row contributes $\sum_{u<v} |c_i(u) - c_i(v)| = (n+1)n(n-1)/6$. The value $3I(L)$ is always an integer. As in the literature, we write imbalances as $x.0$, $x.3$ and $x.6$ for $x$, $x + \frac{1}{3}$ and $x + \frac{2}{3}$. The goal is a rectangle of minimum imbalance for given $k$ and $n$.

## 3 Method

**Search space and move.** The state is $k$ permutations of the symbols, one per row. We fix the first row to the identity. This loses no generality, because a relabelling of the symbols does not change the imbalance. A move picks a row $i \ge 2$ and two columns $j_1 \ne j_2$, and it swaps the symbols $a = L[i][j_1]$ and $b = L[i][j_2]$. Rows stay permutations, but the swap can create a repeated symbol in a column.

**Incremental evaluation.** We keep the matrix $D$ of all pair distances. A swap in row $i$ moves only $a$ and $b$. So only the $2(n-2)$ distances $d(a,x)$ and $d(b,x)$ with $x \notin \{a,b\}$ change, and $d(a,b)$ stays the same. For each such $x$, with $p_x = c_i(x)$,

$$d'(a,x) = d(a,x) - |j_1 - p_x| + |j_2 - p_x|, \qquad d'(b,x) = d(b,x) - |j_2 - p_x| + |j_1 - p_x|.$$

The search therefore computes the exact change of $3I(L)$ in $O(n)$ integer operations. A recomputation from scratch costs $O(kn^2)$.

**Column conflicts.** Let $V$ be the number of excess symbol occurrences over all columns. Per-column counters give $V$ in $O(1)$ time per move. The search minimizes $3I(L) + W \cdot V$, and it records only rectangles with $V = 0$. The weight $W$ starts at $k$. Every $2^{16}$ moves, the search multiplies $W$ by 1.02 if the current state has conflicts and by 0.99 if it has none, and it clamps $W$ to $[1, 4n^2]$. The penalty matters most when $k$ is close to $n$. There, almost every swap creates a conflict, so a search that effectively forbids conflicts stalls. Our first version used a large fixed penalty ($W = 4n^2$). In

10-second runs on $6 \times 7$ and $7 \times 7$, it never visited a conflict-free rectangle other than its start. With the adaptive weight, 20-second runs reach the known optima of both instances (20.0 and 18.6).

**Annealing schedule.** The search starts from the cyclic rectangle $L[i][j] = i+j \bmod n$. It accepts a move with change $\Delta$ if $\Delta \leq 0$, and otherwise with probability $e^{-\Delta/T}$. The temperature $T$ starts at $T_0 = 2k$ and falls by a factor of 0.9999995 after every move, with a floor of 0.05. When the best rectangle does not improve for $4 \cdot 10^7$ moves, the search resets $T$ to $T_0$. Each run is single-threaded and stops after a fixed CPU time. Before it reports its best rectangle, it recomputes the imbalance from scratch. The implementation has about 130 lines of C.

## 4 Experiments

**Setup.** We use the instances in Table 1 of Voboril et al. [2025], with $8 \leq n \leq 13$ and $2 \leq k \leq n-1$. This gives 51 instances. We omit squares ($k = n$), because different structure governs that case [Xia et al., 2026]. The record of each instance is the best value reported there. It already includes earlier results and four-hour Gurobi runs. We used an Apple M3 laptop (8 cores, 16 GB) and ran eight single-threaded runs at a time. Each instance got one run with a budget of 1 minute and one run with 5 minutes, with different seeds. The larger instances ($k \geq 4$, $n \geq 9$) also got one 10-minute run. We report the best rectangle per instance, so no instance used more than 16 CPU minutes.

**Verification.** An independent Python program checks every reported rectangle. We wrote it from the definition above. It uses exact rational arithmetic and shares no code with the solver. To make sure that it computes the published quantity, we ran it on all eleven rectangles printed in [Peruvemba Ramaswamy and Szeider, 2023, Voboril et al., 2025]. These include the eight CP 2025 records of their Figure 6, up to $12 \times 13$. The checker reproduces every reported imbalance exactly.

**Results.** Table 1 shows the outcome. The search improves 33 instances, matches 18, and is worse on 0. All improved instances have $n \geq 9$ and $k \geq 3$. The search matches the record on all instances with $n = 8$ or $k = 2$, on all instances with $k = 3$ except $3 \times 12$, and on $4 \times 10$, $5 \times 10$ and $6 \times 12$. Several of these records are proven optimal [Peruvemba Ramaswamy and Szeider, 2023]. The gains grow with the size of the instance (Figure 1a), up to 39.4% for $12 \times 13$. The one-minute runs alone beat the record on 33 instances. Longer runs mainly widen the margin. Over the 35 instances that got all three budgets, the mean reduction relative to the record is 13.3% after one minute, 15.8% after five minutes and 16.8% after ten minutes.

**Speed.** To measure speed directly, we ran the search on every improved instance with 5 further seeds. Each run stopped as soon as its imbalance fell below the record, with a cap of five CPU minutes (Figure 1b). All 165 runs beat the record. The median time is 0.12 seconds, and the slowest run took 143 seconds. The median per instance is below one second on 23 of the 33 instances. It is below one minute on all of them, with a maximum of 54 seconds for $9 \times 10$. For comparison, each approach of Voboril et al. [2025] runs for 4 to 9 hours per instance, after a 4-hour Gurobi baseline. They also report that their incremental warmstart approach is still 25–60% worse than the baseline after one minute.

Peruvemba Ramaswamy and Szeider [2023] report the SAT solving time for four rectangles that set records at that time. The times range from 32 minutes ($3 \times 10$, imbalance 84) to 5.5

| $n\backslash k$ | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 8 | 40.0 | 36.0 | 32.0 | 30.0 | 24.0 | 28.0 | | | | | |
| 9 | 65.3 | 56.0 | **53.3** ~~56.0~~ | **48.6** ~~52.6~~ | **40.0** ~~48.0~~ | **43.3** ~~51.3~~ | **42.6** ~~53.3~~ | | | | |
| 10 | 92.0 | 80.0 | 78.6 | 66.6 | **60.0** ~~70.0~~ | **58.0** ~~72.0~~ | **58.6** ~~82.6~~ | **64.0** ~~70.0~~ | | | |
| 11 | 124.0 | 114.0 | **104.0** ~~112.0~~ | **94.0** ~~116.0~~ | **88.0** ~~108.0~~ | **84.0** ~~118.0~~ | **78.0** ~~114.0~~ | **84.0** ~~120.0~~ | **100.0** ~~110.0~~ | | |
| 12 | 168.0 | **152.0** ~~154.0~~ | **149.3** ~~154.6~~ | **140.0** ~~145.3~~ | 120.0 | **121.3** ~~164.0~~ | **116.6** ~~148.0~~ | **106.0** ~~154.0~~ | **124.0** ~~174.6~~ | **136.6** ~~147.3~~ | |
| 13 | 218.6 | 198.0 | **201.3** ~~211.3~~ | **191.3** ~~210.6~~ | **166.0** ~~204.0~~ | **164.0** ~~203.3~~ | **160.0** ~~220.6~~ | **152.0** ~~214.0~~ | **154.6** ~~240.6~~ | **168.0** ~~224.0~~ | **166.0** ~~274.0~~ |

**Table 1.** Best known imbalance of $k \times n$ Latin rectangles. Bold: new value from this work, with the record [Voboril et al., 2025] struck through below it. Plain: record that our search matches.

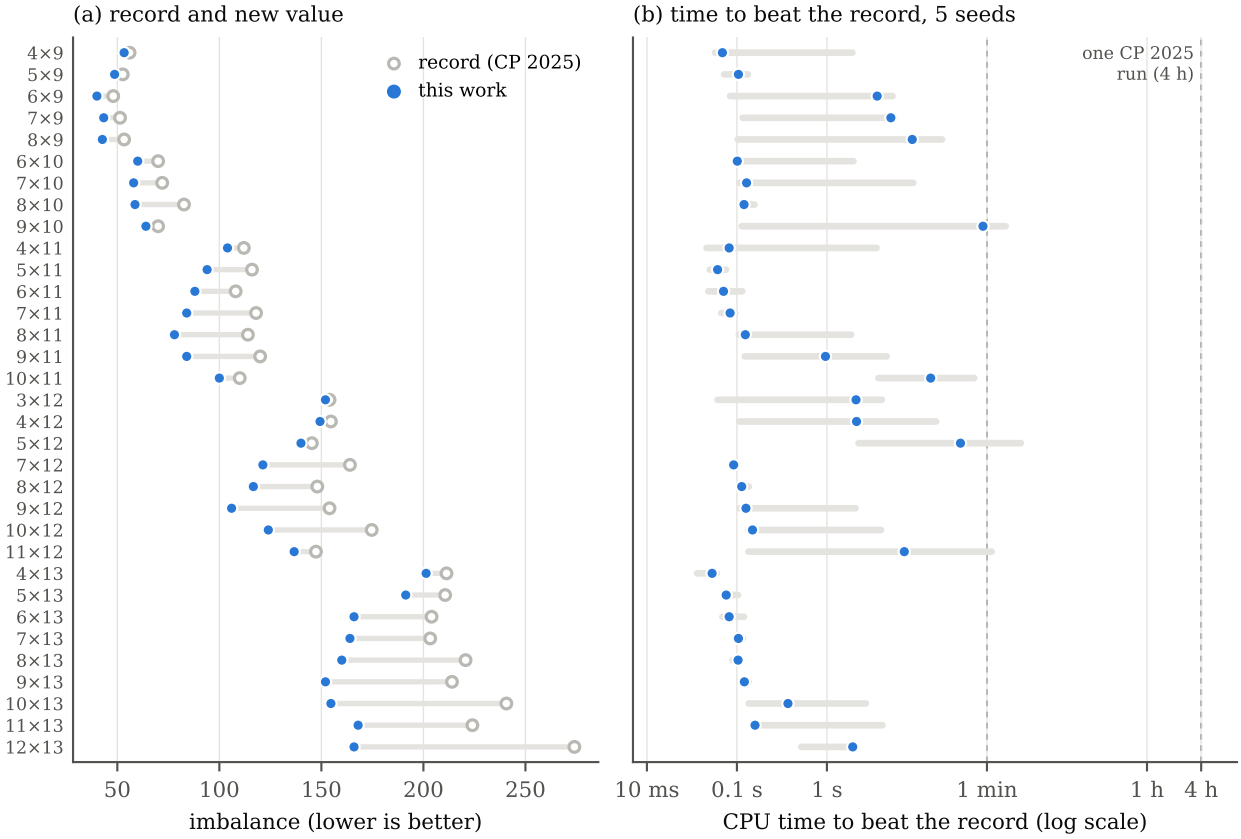



**Figure 1.** All 33 improved instances. (a) Record (open circle) and new imbalance (filled). (b) CPU time until the search beats the record, on one core: median (dot) and range (bar) over 5 seeds, log scale. The dashed lines mark one minute and one 4-hour run of one CP 2025 approach.

hours ($6 \times 11$, imbalance 124). Figure 2 shows all four. In every run, our search reaches the same imbalance within 0.07 seconds. By the median, it is at least 43,636 times faster.

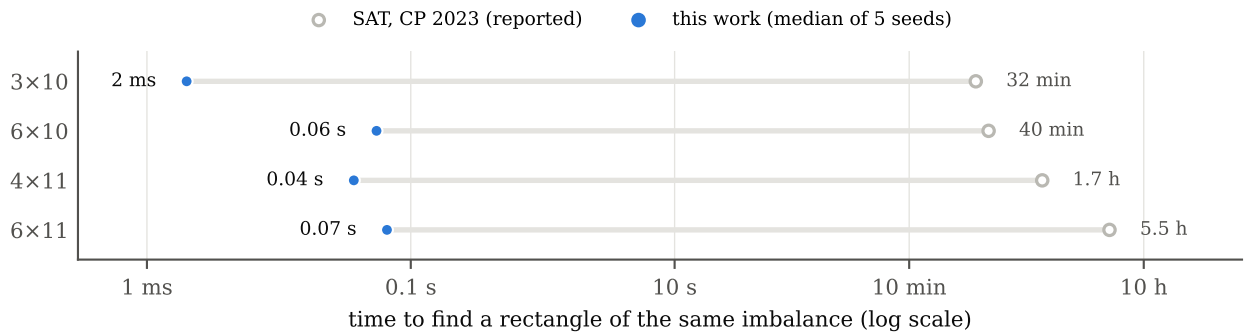



**Figure 2.** Time to find a rectangle with the imbalance that the SAT approach of Peruvemba Ramaswamy and Szeider [2023] reports, next to its reported solving time. Ours: median of 5 seeds. Log scale.

## 5 Discussion

**Why does a simple method do so well?** We have not tested this, but the structure of the problem suggests an explanation. The objective is a sum of $\binom{n}{2}$ absolute deviations. A MIP model must linearize every one of them. Its LP relaxation can make all pairs look nearly balanced at once, so it likely gives the solver little guidance. Streamliners help the solver because they restrict it to a structured subspace, but the best rectangles need not lie in that subspace. Local search avoids both problems. The search updates the objective exactly and cheaply, the swap neighbourhood is natural, and the penalty lets the search cross infeasible states. The problem suits annealing unusually well, and we do not claim that the comparison carries over to other problems.

**What this means for LLM-based methods.** The previous records came from a pipeline that combines Gurobi, five LLMs and an evolutionary search over streamliners. This pipeline runs for hours per instance. A simulated-annealing search of about 130 lines of C improves 33 of these records and matches the other 18, within seconds. We do not read this result as a verdict on LLM-generated streamliners. Their goal is generality across constraint models, not the best solver for one problem. A general method can lose to a specialized one on a single benchmark. We do read the result as a reminder about baselines. Local search for balanced Latin designs goes back at least to Smith et al. [2005], and Díaz et al. [2017] included it among their methods. Yet the two recent studies of this benchmark [Peruvemba Ramaswamy and Szeider, 2023, Voboril et al., 2025] compare only against exact solvers and earlier records. New methods for this benchmark, LLM-based or not, should report a strong problem-specific baseline next to the exact solvers.

**How fair is the speed comparison?** The gap spans four to five orders of magnitude, so we state its caveats plainly.

1. The hardware differs. The earlier work used Intel Xeon E5-2640 v4 servers from 2016. We used an Apple M3 laptop from 2023. Its single-core speed is higher by a small factor, not by orders of magnitude.

2. CP 2025 does not report when within its runs it found each record. We therefore compare our time to beat a record with the length of their runs, not with their time to reach the record.

3. The CP 2023 times come from a complete SAT solver on a decision encoding. These instances are satisfiable, so the solver had to find a rectangle, which is the same task as ours. Its encoding, however, is built to prove infeasibility as well.

4. We did not rerun either pipeline ourselves.

Together, these caveats are unlikely to explain a gap of this size.

**Limitations.** The new values are upper bounds. We prove nothing about optimality, and the size of the improvements suggests that the remaining gap to the optimum may still be large. The final records come from one run per instance and budget. Only the speed experiment uses several seeds. Finally, the benchmark stops at $n = 13$, and larger instances are an obvious next step.

## 6 Conclusion

A short, standard simulated-annealing search improves 33 of the 51 best known imbalance values for balanced Latin rectangles in the recent literature, and it matches the rest. It beats each record within seconds on a laptop, where earlier methods ran for hours on a server. Natural next steps follow from this.

- Prove lower bounds or optimality for the improved instances, for example with the SAT approach of Peruvemba Ramaswamy and Szeider [2023].
- Extend the benchmark to larger instances.
- Compare exact, LLM-guided and local-search methods on equal hardware.

**Data and code.** Rectangles, checker, solver and scripts: `https://github.com/BrianLi009/Balanced-Latin-Rectangles`.

## References


Mateo Díaz, Ronan Le Bras, and Carla P. Gomes. In search of balance: The challenge of generating balanced Latin rectangles. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2017)*, volume 10335 of *Lecture Notes in Computer Science*, pages 68–76. Springer, 2017. doi: 10.1007/978-3-319-59776-8_6.

Fred Glover and Jin-Kao Hao. The case for strategic oscillation. *Annals of Operations Research*, 183(1):163–173, 2011. doi: 10.1007/s10479-009-0597-1.

Carla P. Gomes and Meinolf Sellmann. Streamlined constraint reasoning. In *Principles and Practice of Constraint Programming (CP 2004)*, volume 3258 of *Lecture Notes in Computer Science*, pages 274–289. Springer, 2004. doi: 10.1007/978-3-540-30201-8_22.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

Ronan Le Bras, Carla P. Gomes, and Bart Selman. From streamlined combinatorial search to efficient constructive procedures. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, pages 499–506, 2012. doi: 10.1609/aaai.v26i1.8147.

Vaidyanathan Peruvemba Ramaswamy and Stefan Szeider. Proven optimally-balanced Latin rectangles with SAT (short paper). In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *LIPIcs*, pages 48:1–48:10. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPIcs.CP.2023.48.

Casey Smith, Carla P. Gomes, and Cèsar Fernández. Streamlining local search for spatially balanced Latin squares. In *Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence (IJCAI-05)*, pages 1539–1540, 2005.

H. M. van Es and C. L. van Es. Spatial nature of randomization and its effect on the outcome of field experiments. *Agronomy Journal*, 85(2):420–428, 1993. doi: 10.2134/agronj1993.00021962008500020046x.

Florentina Voboril and Stefan Szeider. Improving constraint models with LLM agents, 2026. arXiv:2608.08127.

Florentina Voboril, Vaidyanathan Peruvemba Ramaswamy, and Stefan Szeider. Balancing Latin rectangles with LLM-generated streamliners. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 36:1–36:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi: 10.4230/LIPIcs.CP.2025.36.

Hai Xia, Carla P. Gomes, Bart Selman, and Stefan Szeider. Agentic neurosymbolic collaboration for mathematical discovery: A case study in combinatorial design, 2026. arXiv:2603.08322.